

UML, или Unified Modeling Language, — это унифицированный язык моделирования. Его используют, чтобы создавать диаграммы и схемы для визуализации процессов и явлений.

Слово «унифицированный» означает, что схемы на UML будут понятны всем, кто знаком с ним. То есть у языка есть определённые правила, по которым применяются все стрелочки, кружки и квадраты.

Представьте, что у нас есть простой сайт, который работает на сервере. К нему подключена база данных, где хранится информация о посетителях и их действиях. Когда пользователь заходит на сайт, он сначала отправляет запрос на сервер, а тот делает запрос в базу данных и возвращает информацию обратно пользователю. И так происходит до тех пор, пока не будет закрыта страница сайта.

Визуально такое взаимодействие можно представить так:

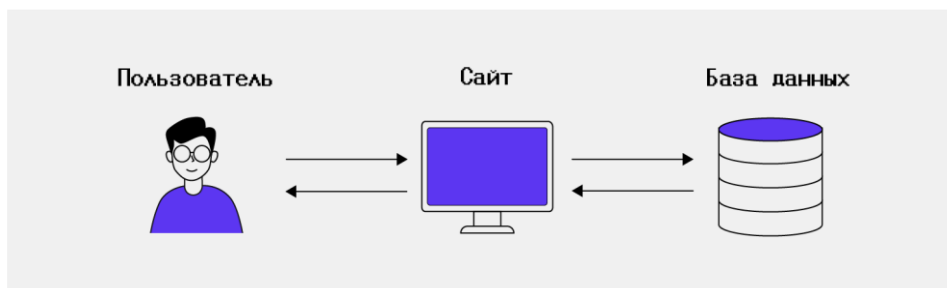


Схема простая: пользователь → сайт → база данных. И в обратном порядке. Только что мы описали процесс — как пользователь взаимодействует с нашим сайтом. Правда, сделали это в очень упрощённом виде и не по правилам языка UML.

Для чего используют язык UML

UML нужен, чтобы превращать абстракции в визуальные модели и схемы.

Поэтому применять его можно во многих областях:

- в программировании — чтобы наглядно видеть связи между классами и другими частями приложения или чтобы построить карту поведения пользователя на сайте;
- в дизайне — чтобы создавать интерфейсы и понимать, как пользователи будут взаимодействовать с ними;

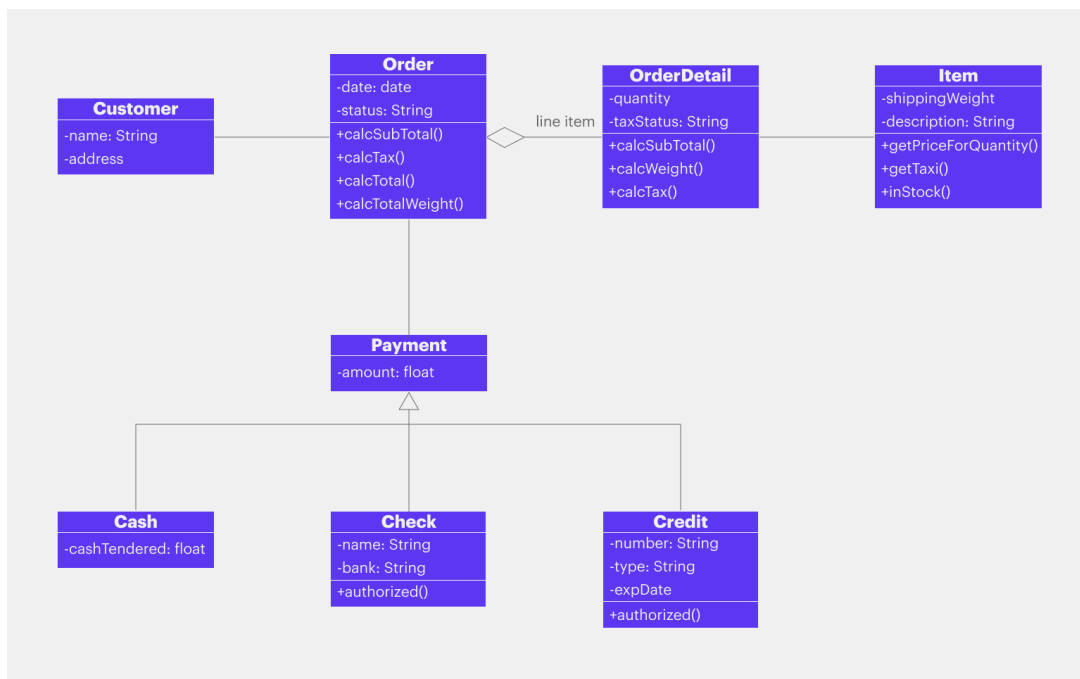
- в бизнесе — чтобы визуально представлять, как работают бизнес-процессы или ведётся документооборот в организации.

При этом процессы можно проектировать с нуля или брать уже существующие и описывать в виде UML-диаграмм. Например, вот так схематично выглядит устройство базы данных музыкального сервиса:



К пользователю привязаны две таблицы — с музыкантами, на которых он подписан, и с понравившимися ему аудиозаписями

Базы данных могут быть сложнее, и для работы с ними как раз и пригодится язык UML. Например, вот как выглядит диаграмма для интернет-магазина:



Эта диаграмма описывает базу данных интернет-магазина

Теперь поговорим о преимуществах и недостатках UML, а затем начнём уже наконец разбирать диаграммы.

Какие преимущества и недостатки у UML

Нарисовать схему может любой — для этого даже не нужен никакой язык моделирования. Достаточно графического редактора. Но если вы хотите, чтобы эти схемы понимали не только вы, но и другие разработчики, то стоит присмотреться именно к UML.

- **Универсальность.** У каждого элемента в UML-диаграммах строго определённое значение, которое не меняется в зависимости от отрасли, страны или предпочтений разработчика. Поэтому если вы знаете базовый синтаксис, то сможете разобраться практически в любой схеме. Это как математические теоремы: их понимают одинаково во всём мире.
- **Простота и визуализация.** Любую информацию легче понять, если она представлена в виде картинки, а UML как раз про это. В диаграммах используют знакомые всем объекты: фигуры и стрелки. По ним легко понять, как устроена структура целиком, даже если вы не особо знакомы с языком.
- **Распространённость.** UML — понятный и простой язык, поэтому его используют во многих областях, например в IT, инженерии и менеджменте. При этом диаграммы из одной сферы смогут без особых усилий понять и в другой, хотя нюансы, конечно, есть.
- **Возможность автоматизации и реверс-инжиниринга.** Язык поддерживает много инструментов для автоматизации моделирования. Например, можно генерировать код прямо из UML-диаграмм — для этого есть такие инструменты, как [Umbrello](#). Это же помогает, когда нужно провести [ревёрс-инжиниринг](#) и понять, как работает система без документации.

Теперь о неприятном — о недостатках языка UML. Они тоже есть, и об этом точно стоит помнить, прежде чем использовать его для ваших проектов.

- **Объёмная документация.** Если решитесь изучить язык, практически сразу же наткнётесь [на его документацию](#). И она удивит вас тем, что там 700 страниц... Хотя читать всю необязательно — достаточно обращаться к ней, когда возникнут вопросы об элементах.
- **Сложность использования и поддержки проектов.** Некоторые инструменты для работы с UML могут показаться сложными или недостаточно интуитивными. А это добавляет трудности в освоении и использовании.
- **Семантическая неоднозначность.** Язык не всегда придерживается строгих обозначений, и это усложняет работу с ним. Некоторые элементы можно интерпретировать по-разному, что приводит к проблемам интерпретации.

Особенно ярко это видно в диаграммах активности и последовательности. Но о них дальше.

- Несовместимость с мелкими проектами. UML разработали, чтобы моделировать большие и сложные системы. Поэтому использовать его для небольших — это избыточно и неэффективно. Так вы только усложните разработку и потратите ресурсы на менее значимые вещи.

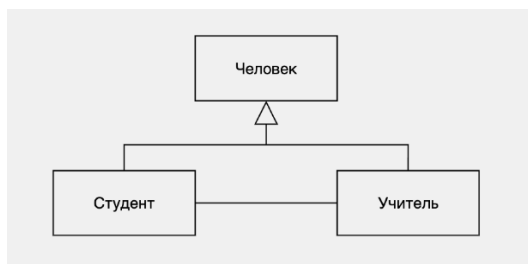
Плюсов у языка достаточно, и они переешивают минусы, поэтому использовать его точно стоит. Теперь перейдём к самим диаграммам, чтобы понять, как они устроены.

Что такое UML-диаграмма

UML-диаграмма — это просто схема, которую сделали по правилам языка UML. Обычно в ней есть элементы и связи между ними, а также дополнительные объекты, которые помогают лучше понять диаграмму.

С помощью UML-схем можно описывать разные процессы и явления. Это мы уже усвоили. Примеры таких диаграмм — зависимости частей приложения друг от друга, описание классов в коде, планировка магазина или архитектура ноутбука.

Вот так выглядит простая диаграмма, которая описывает связи между классами в коде:



На диаграмме есть три класса: Студент, Учитель и Человек. Все они как-то относятся друг к другу. Сейчас коротко поясним.

Классы Студент и Учитель связаны между собой — это показано обычной линией. Связаны — значит, что они как-то взаимодействуют: например, вызывают методы друг друга.

Классы Студент и Учитель наследуют свойства от класса Человек.

Наследуют — значит берут этот класс за основу. Например, у Человека могут быть свойства: рост, вес и имя. А все классы, которые наследуются от него,

сразу же получают эти свойства. Наследование показано незакрашенной стрелкой.

Все диаграммы строятся по главному принципу: есть объекты, которые как-то связаны друг с другом. Эта связь может быть разной — например, наследование или обычное взаимодействие между объектами.

Из чего состоят UML-диаграммы

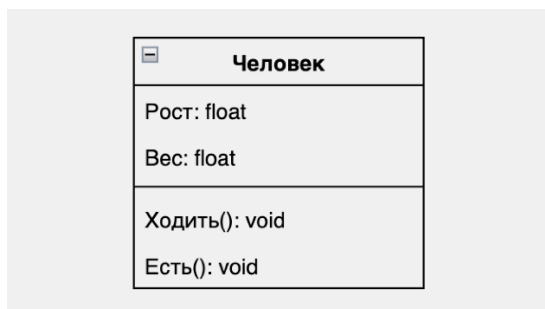
В языке есть десятки элементов для обозначения любых процессов.

Класс

Весь UML основан на принципах объектно-ориентированного программирования. Поэтому дальше мы часто будем говорить о классах, объектах, интерфейсах. Этот подход очень популярен среди разработчиков, и создатели UML решили сфокусироваться именно на нём.

Класс — это структура данных, у которой есть свойства и методы. Из него создаются объекты — копии класса со всеми его внутренностями, но всегда с разными значениями.

У свойств класса есть имена и типы данных. А у методов — тоже имена и типы возвращаемых значений. Выглядит класс так:



У нашего человека есть рост и вес, а из действий он умеет ходить и есть

На прямоугольнике есть три секции: с названием класса, свойствами и методами. Давайте о них подробнее.

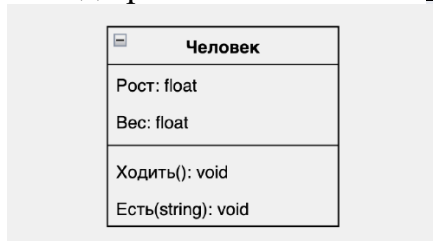
- Название класса — это уникальное имя, по которому мы можем определить, зачем нужен этот класс. В нашем случае, класс Человек — это набор характеристик людей и действия, которые они могут совершать.

- Свойства класса — это переменные класса, которые нужны для чего-то. Например, для человека мы можем перевести его рост и вес в американские обозначения.

- Методы класса — это действия, которые класс может выполнять.

Например, метод Ходить позволяет человеку ходить, а метод Есть — есть. Заметьте, что у свойств и методов есть типы — float и void. Для свойств они показывают, какой тип переменных будет храниться в ней. А для методов — какой тип будет возвращаться после выполнения действия.

Чтобы обозначить метод, после его названия ставят две круглые скобки. А если метод принимает какой-то аргумент, то его тип можно указать внутри скобок:



Теперь в метод Есть() передаётся название еды

Объект

Это копия класса, у которой есть все его свойства и методы. Объекты нужны, чтобы создавать различные версии классов с разными значениями. Например, мы можем создать объект класса Человек:



Для объекта не обязательно указывать свойства, достаточно показать, что этот объект относится к классу Человек. Но о связях мы поговорим дальше

Интерфейс

Это как класс, только что-то абстрактное. Интерфейс можно представить как нечто, что помогает двум другим объектам взаимодействовать друг с другом. Например, интерфейс сайта позволяет пользователю не разбираться, как отправлять HTTP-запросы, а просто нажимать кнопки с понятным названием.

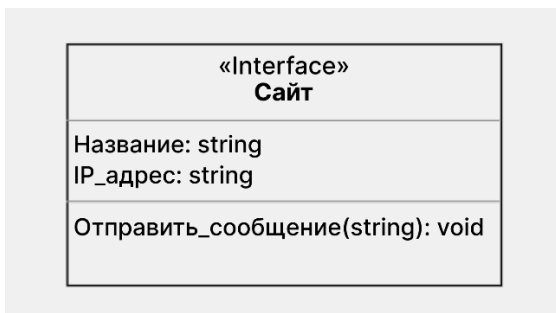
В программировании интерфейсы используют, чтобы показать наличие класса, но без его внутренностей. В UML это выглядит так:



Простой интерфейс для сайта

Для интерфейса нужно обязательно указать слово «interface». Иначе его можно будет спутать с обычным объектом или классом.

Ещё можно указать, какие у интерфейса должны быть свойства и методы, которые обязательно нужно сделать программисту:



Когда создадут класс по этому интерфейсу, нужно будет обязательно добавить эти свойства и метод

Компонент

Это более крупная часть приложения — например, файл или библиотека.

С помощью компонентов можно показать, как взаимодействуют разные части системы между собой. Выглядит он так:

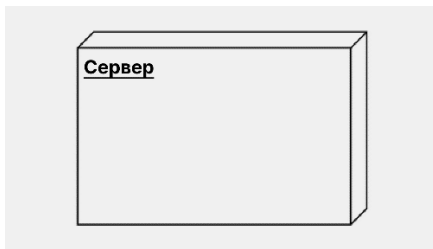


Это главный файл для сайта, который отображает страницу

Узел

Это как компонент, только ещё больше. Пример — сервер или база данных.

Внутри узла может находиться несколько компонентов. А выглядит узел так:

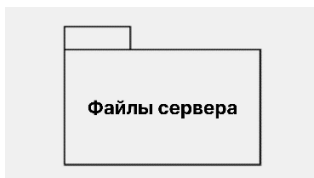


Чем крупнее, тем более объёмна фигура

Отметим одну вроде бы уже очевидную вещь: элементы UML можно вкладывать друг в друга. Получается такая матрёшка, которую мы можем раскрывать и смотреть, как устроена структура на уровень ниже.

Пакет

Это объединение нескольких объектов диаграммы в одну группу. Обычно это нужно, чтобы располагать вместе похожие объекты — например, все файлы, которые относятся к серверной части приложения. Выглядит пакет так:



Внутри — все файлы, что находятся на сервере

Состояние

Это особое состояние системы, через которое она проходит. Например, когда сервер получил запрос и начал его обработку, то он больше не сможет принимать новые запросы, пока не закончит этот.

Состояние выглядит так:



Это состояние, когда пользователь проходит авторизацию

Заметка

Это комментарий для разных объектов в UML. У заметок нет определённой структуры: они нужны просто для уточнений подробностей. Выглядят так:



Человек — это
базовый класс для
всех людей в мире.

Поясняем, зачем нужен класс Человек

Юзкейс

Это действие, которое может выполнить система. Например, приложение может отправить запрос в базу данных или вернуть пользователю эти данные, если они уже пришли. Юзкейс выглядит так:



Погладить кота

Все мы люди, всем хочется погладить кота — даже компьютеру

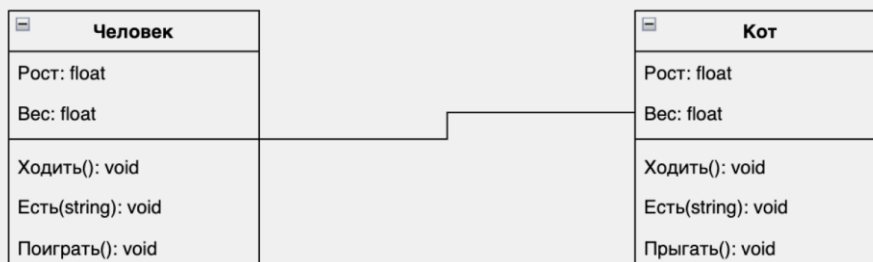
СВЯЗЬ

Это линия, которая показывает, что два объекта связаны между собой. Связь может быть любой — например, два объекта могут вызывать методы друг друга. Обозначается это так:



Да, это обычная прямая линия, без всяких стрелок

А так выглядит связь между двумя классами:



Человек и Животное могут взаимодействовать через методы Поиграть() и Прыгать(). Поэтому два класса связаны

Взаимодействие

Это стрелка, которая показывает, что один объект обращается к другому.

Взаимодействие также указывает, какие данные передаются адресату.

Показывают взаимодействие так:

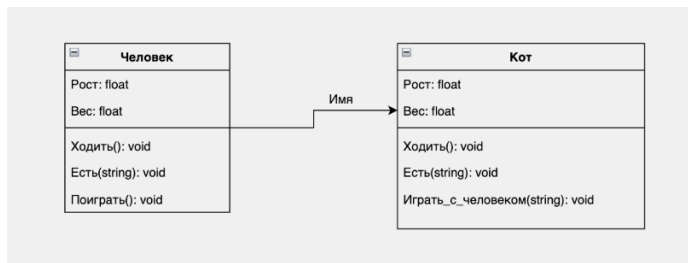


Передаём имя по стрелке слева направо

Сверху над стрелкой указывают, какие данные мы передаём другому элементу.

В нашем случае это чьё-то имя. Но данные могут быть любыми.

Посмотрим, как два класса взаимодействуют между собой:



У класса Животное появился метод Играть_с_человеком(), который принимает имя человека в виде строки. Поэтому два класса взаимодействуют

Зависимость

Это тоже стрелка, но пунктирная, которая показывает, что если мы изменим

один объект, то одновременно изменим и другой. Выглядит зависимость так:



Сверху ещё можно указать, что именно поменяется

Пример зависимости — это градусник и температура. Если меняется

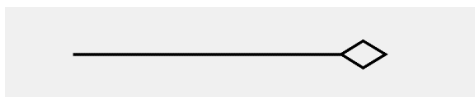
температура, то меняется и значение на градуснике:



Температура изменилась — изменилось и значение на градуснике

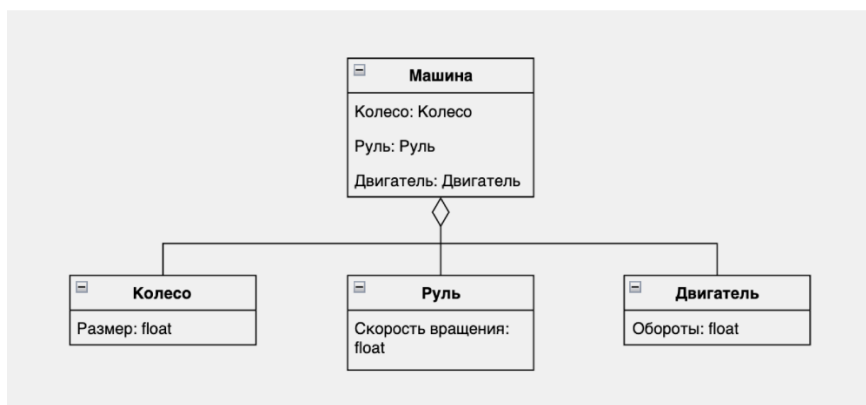
Агрегация

Это указание, что один объект является частью другого. Например, машина, говоря упрощённо, состоит из колеса, руля и двигателя. Обозначается агрегация так:



Слева указывается базовый элемент, а справа — объект, внутри которого есть базовый

Как это выглядит в схеме? Посмотрим агрегацию на примере машины, которая состоит из нескольких частей:

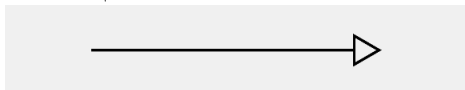


Здесь у машины есть три базовых класса: Колесо, Руль и Двигатель

Обобщение или наследование

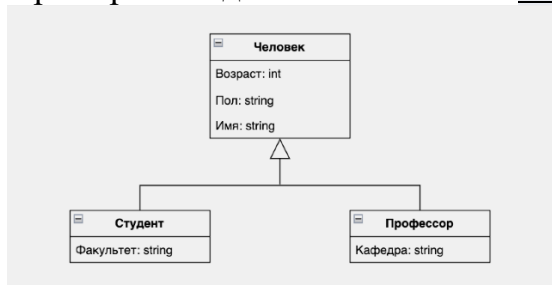
Это стрелка, которая показывает, что один объект наследует свойства другого.

Например, класс Студент может наследовать класс Человек. Выглядит обобщение так:



Справа ставят класс, от которого наследуют, а слева — класс потомок.

Пример наследования — классы Человек, Студент и Учитель:



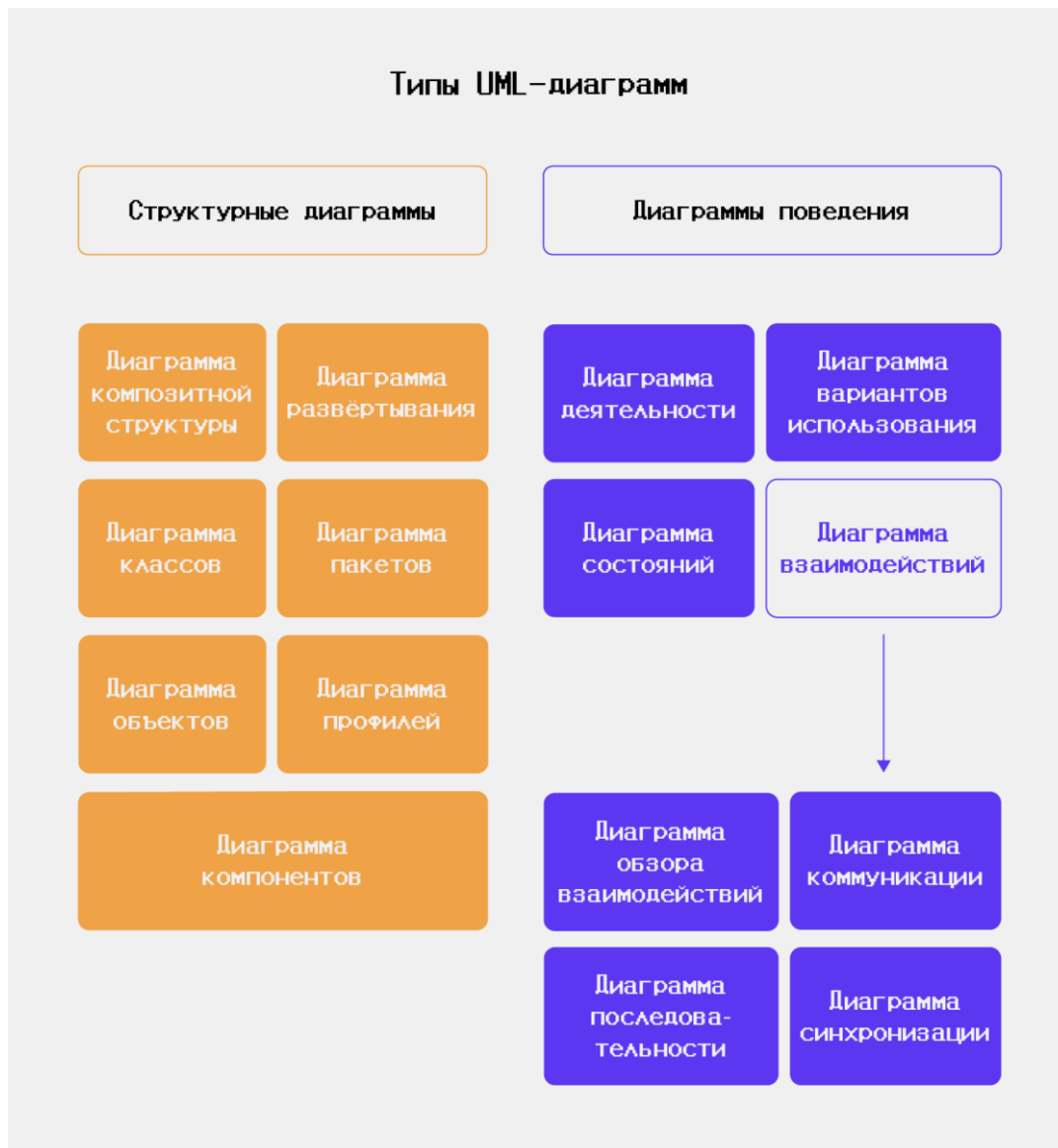
Классы Студент и Учитель наследуют все свойства класса Человек, а также добавляют собственные

Пока закончим знакомство с элементами UML. Всё за один раз не изучить — да это и не нужно. Лучше всего постоянно обращаться к [спецификации UML](#) и находить там нужные объекты.

Типы UML-диаграмм

Все UML-диаграммы делятся два больших типа — структурные и поведенческие. Первые нужны, чтобы описывать структуру объектов и приложений. А вторые показывают, как проходят взаимодействия внутри системы.

Если посмотреть ещё глубже, то структурные и поведенческие диаграммы разделяются на несколько типов поменьше. Все их можно представить в виде такой схемы:

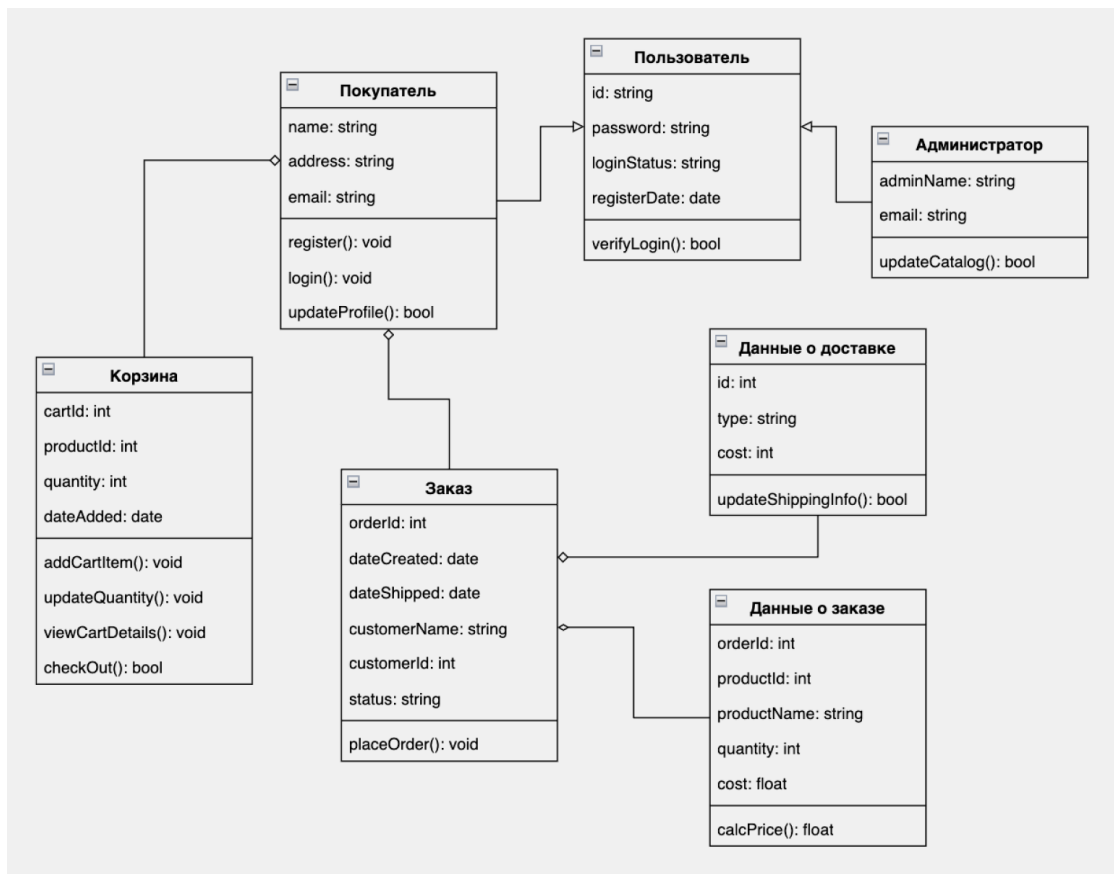


Получается, всего 14 видов разных диаграмм. Но запоминать все уж точно не нужно

Чаще всего используют лишь шесть из них. Поэтому давайте познакомимся с ними поближе.

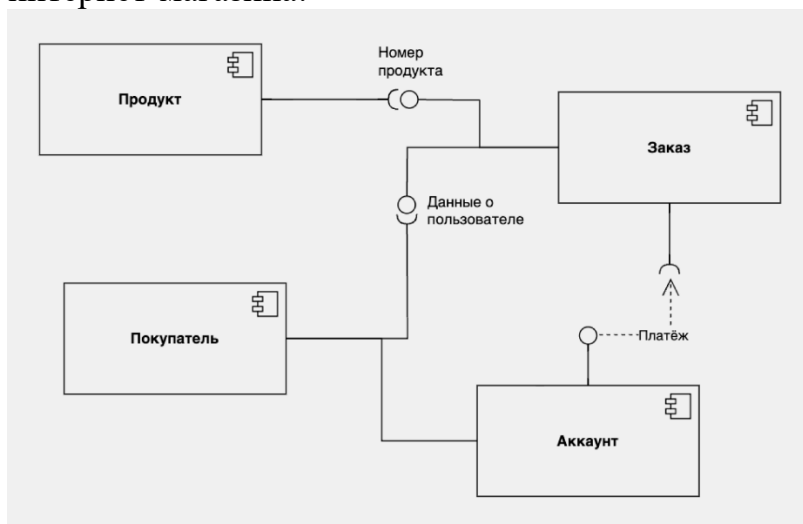
Структурные диаграммы

Диаграмма классов. Описывает структуру и связи между объектами и классами. Такая диаграмма помогает понять, какие классы от кого наследуются и с кем взаимодействуют. Выглядит она так:



На диаграмме изображены классы для интернет-магазина

Диаграмма компонентов. Показывает, какие компоненты есть в системе и как они взаимодействуют между собой. Пример такой диаграммы — приложение интернет-магазина:

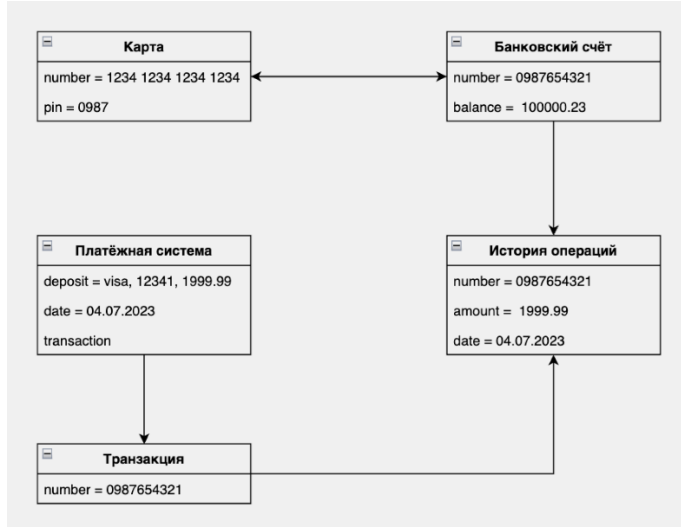


Напомним, что прямоугольники с этим символом в правом верхнем углу — это компоненты

Видим, что появились новые обозначения в виде круга и полукруга. Они указывают на то, что для взаимодействия компонентов нужен интерфейс. А ещё

есть круг, полукруг, стрелка между ними и надпись «Платёж». Это значит, что для взаимодействия компонентов должен произойти платёж.

Диаграмма объектов. Нужна, чтобы показать состояние элементов в конкретный момент времени. Это помогает лучше понять, как моделировать объекты.

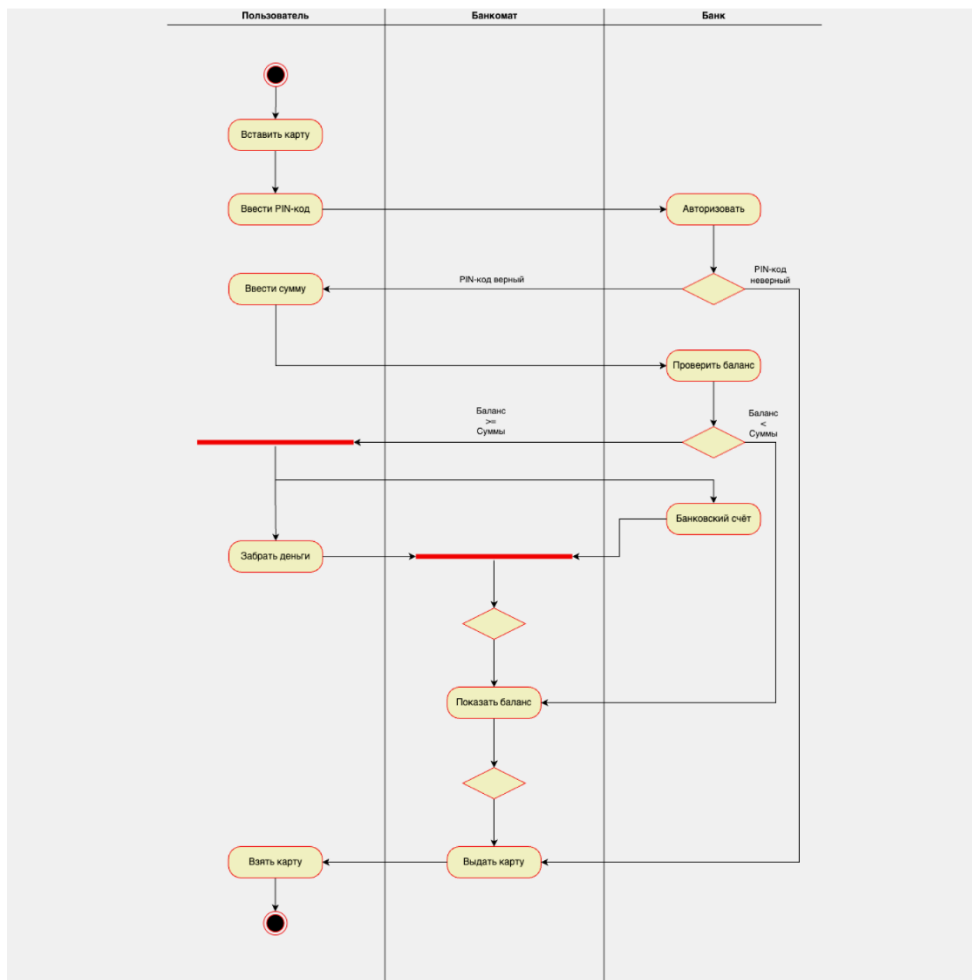


Так выглядит диаграмма объектов для оплаты чего-то в интернете.

Поведенческие диаграммы

Диаграмма деятельности. Показывает всю последовательность работы системы.

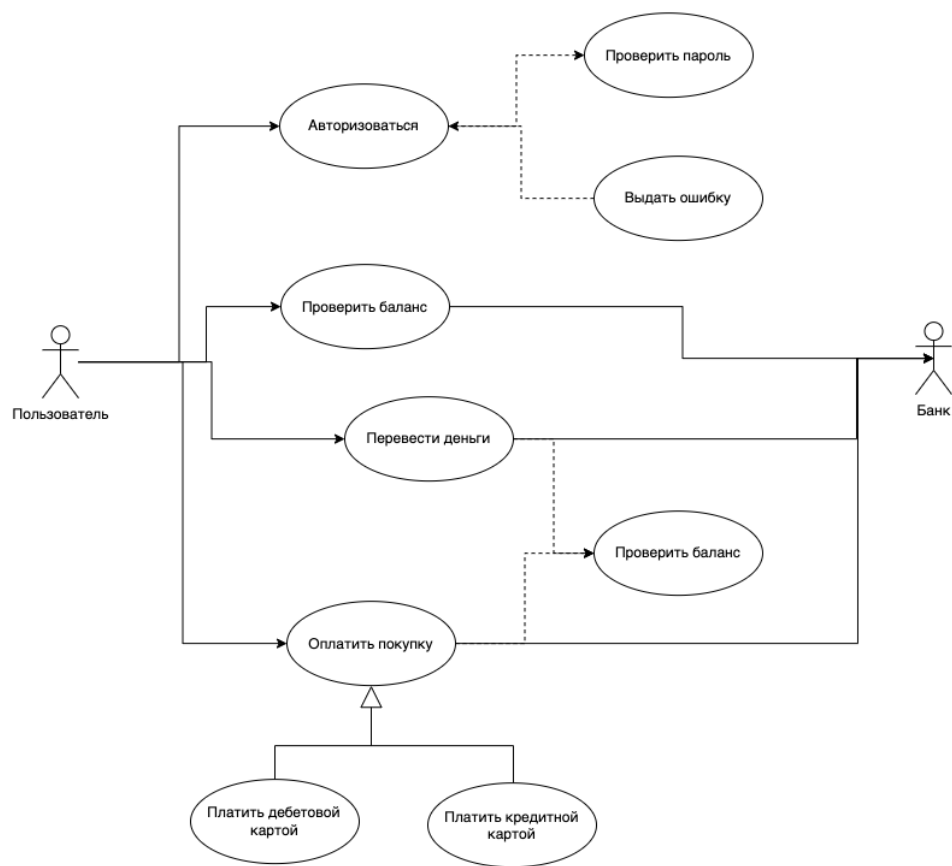
Диаграмма выглядит как прямые указания приложению, что нужно делать и когда.



На диаграмме изображена схема показывающая, как пользователь взаимодействует с банкоматом и как банкомат связывается с банком

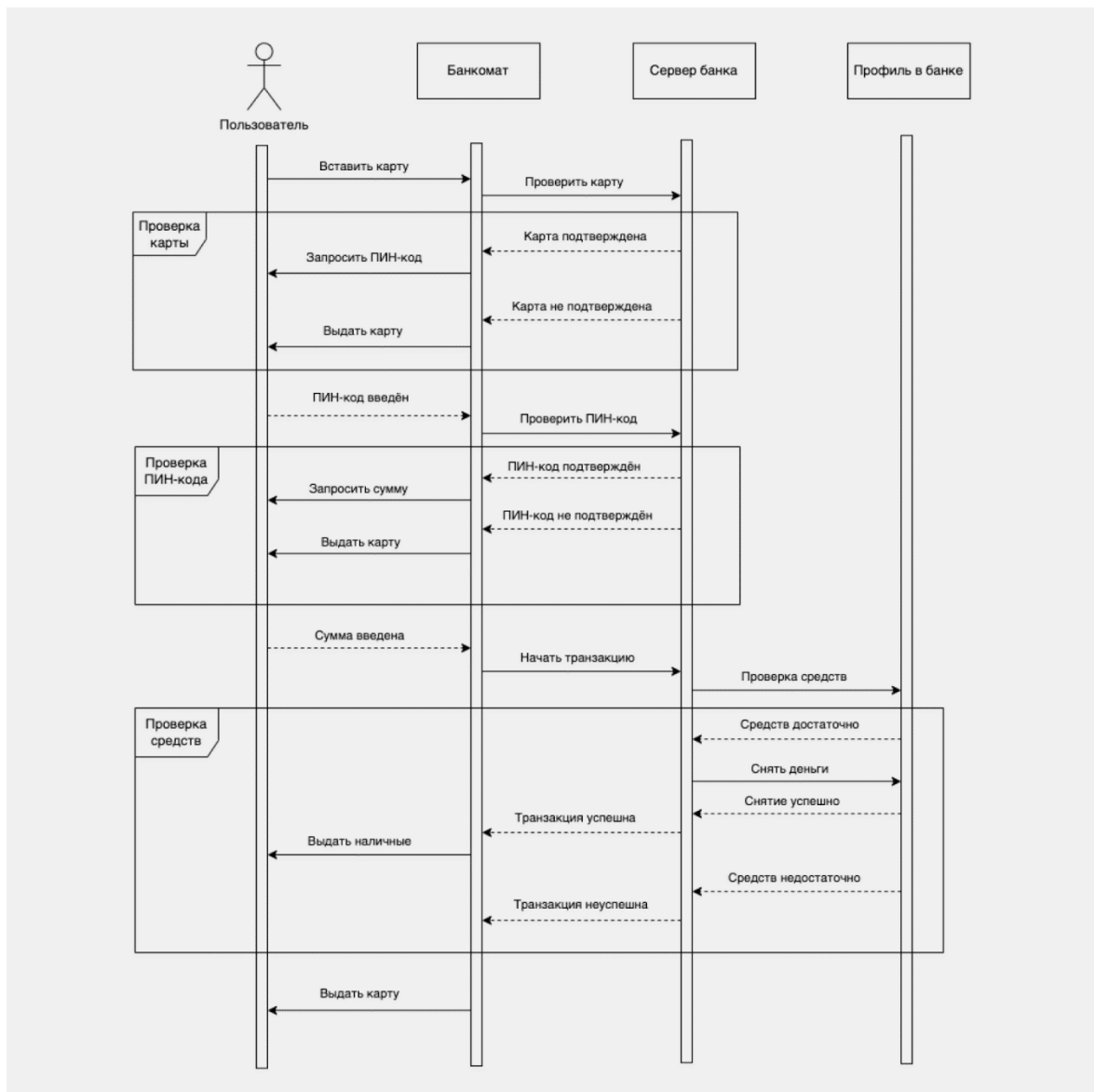
Видим, что на диаграмме ещё появились ромбики. Это условие, от которого зависит, к какому следующему действию мы перейдём.

Диаграмма вариантов использования. Нужна, чтобы показать, как пользователи взаимодействуют с системой. На этой диаграмме можно указать все сценарии использования, а также показать, как перейти из одного сценария в другой.



Здесь показан процесс взаимодействия пользователя с приложением банком

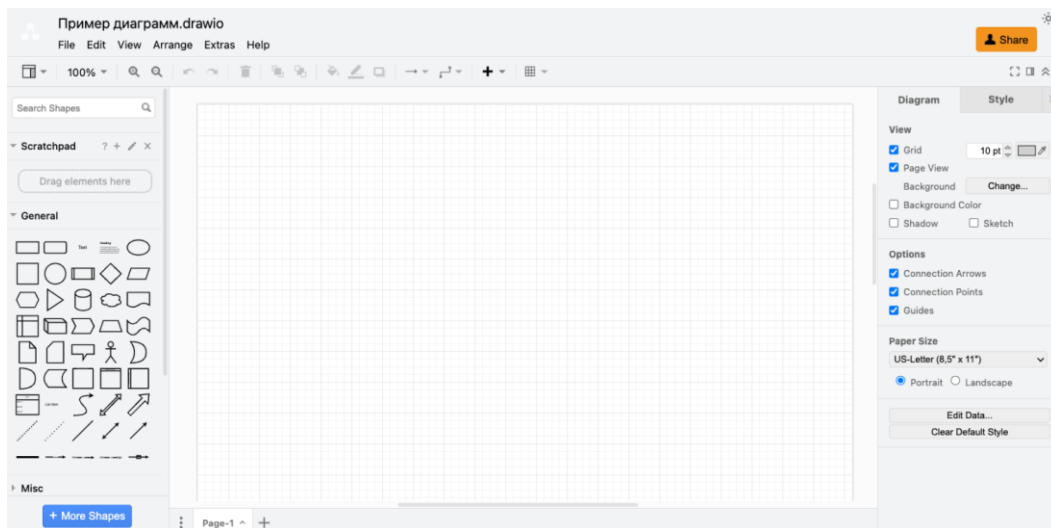
Диаграмма последовательности. Описывает все возможные действия, которые может выполнить пользователь. Она чем-то похожа на диаграмму вариантов использования.



Тут показан полный процесс взаимодействия пользователя с банкоматом и все возможные сценарии

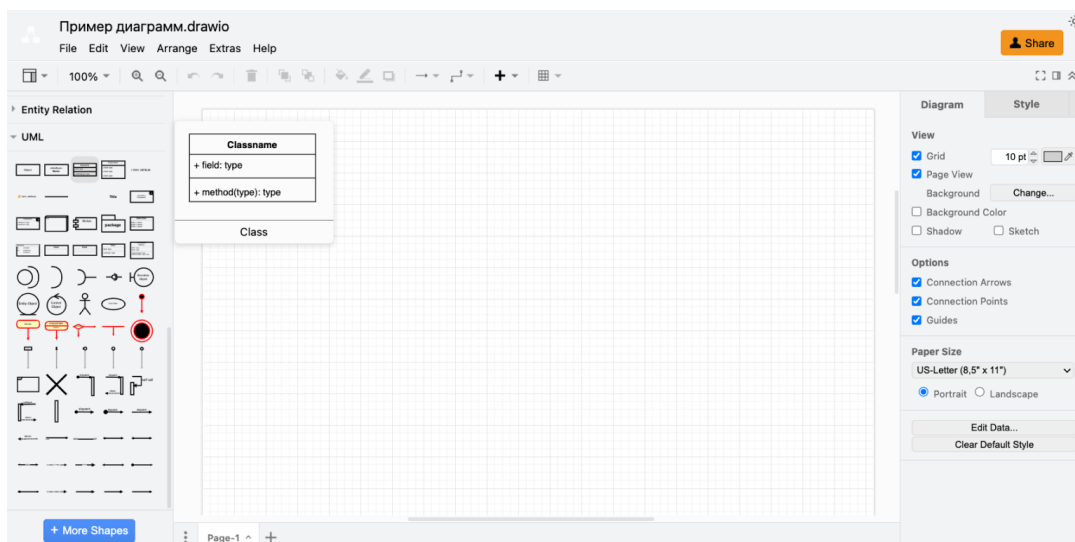
Как построить UML-диаграмму

Чтобы построить UML-диаграмму, можно даже не устанавливать дополнительные приложения — всё уже доступно в браузере. Для нашей статьи мы делали все схемы в сервисе draw.io. Вот как он выглядит:



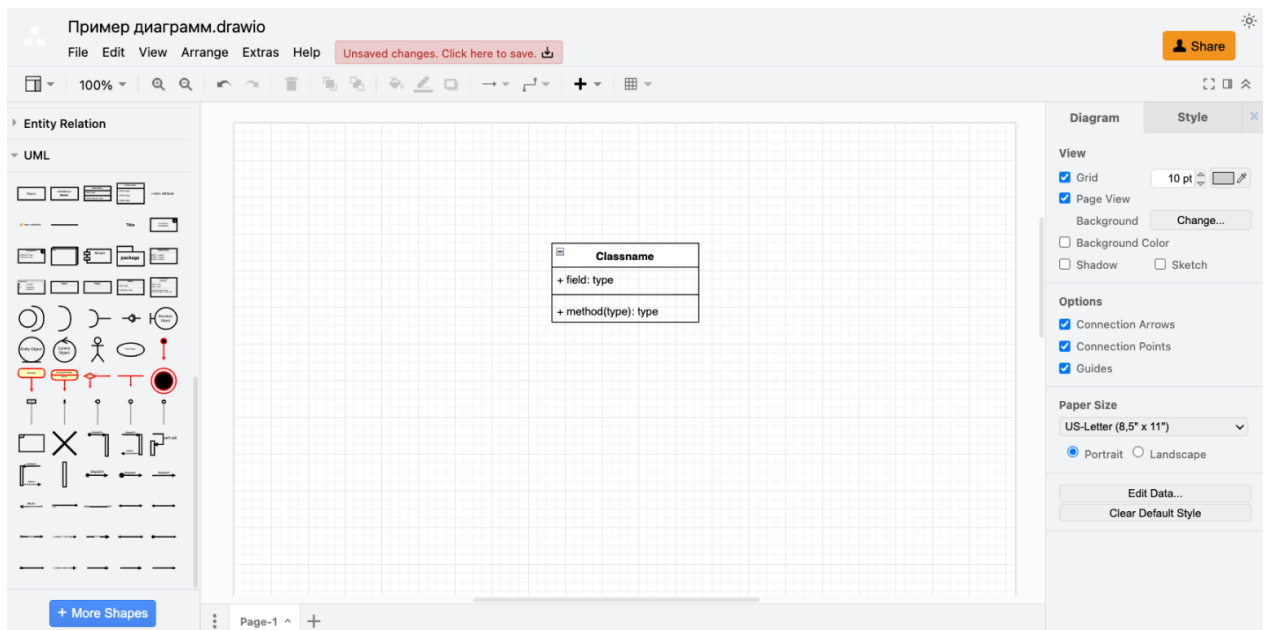
Пустая страница инструмента draw.io

В сервисе есть все нужные для UML элементы: классы, объекты, связи и прочие. Находятся они в левом меню:



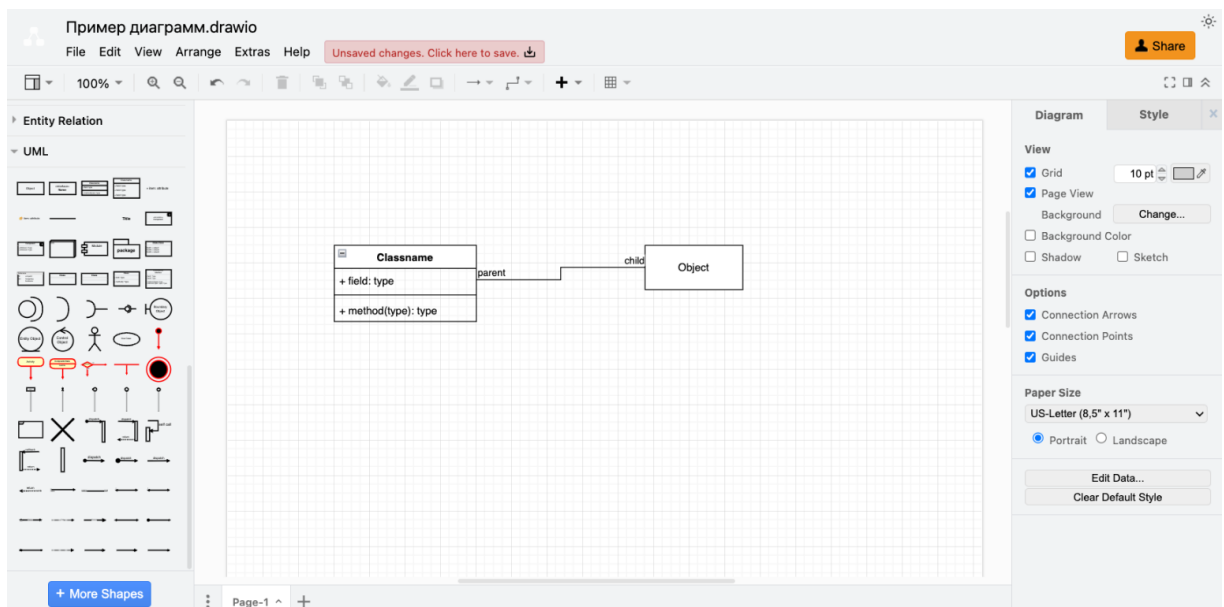
Элементы берём из левого меню

Чтобы добавить элемент, нужно просто перетащить его в центральную область:



Интерфейс инструмента draw.io

А чтобы сделать связь, можно воспользоваться стрелками из меню:



Так добавляем элементы и соединяем их

Кроме этого сервиса есть ещё много других. Вот список некоторых из них:

- [Lucidchart](#) — популярный инструмент для создания диаграмм. Он предлагает широкий выбор готовых шаблонов и символов, а также возможность совместной работы в реальном времени.
- [Visual Paradigm](#) — полнофункциональный инструмент для моделирования и создания UML-диаграмм. Он предоставляет множество функций и возможностей для анализа и разработки систем.

- [Creately](#) предлагает простой в использовании веб-интерфейс для создания UML-диаграмм. Он содержит множество шаблонов и символов, а также поддерживает совместную работу и комментирование.
- [PlantUML](#) — представляет собой инструмент для создания диаграмм, основанный на текстовом описании с использованием языка UML. Он позволяет создавать UML-диаграммы путем написания текстового кода, что делает его гибким и простым в использовании.

Что запомнить

Давайте резюмируем всё, что мы узнали о языке UML:

- UML — это унифицированный язык для создания схем. Унифицированный — значит, что если знать базовый синтаксис, то можно понять любую схему.
- Язык нужен, чтобы визуально описывать явления и процессы. Например, как работают приложения или как идут бизнес-процессы в компании.
- У UML несколько главных преимуществ: универсальность, простота и визуализация, распространённость, а также возможность автоматизации и реверс-инжиниринга.
- UML-диаграммы — это схемы, которые сделаны по правилам языка UML.
- Диаграммы состоят из разных базовых элементов — например, классов, объектов, узлов и связей между ними.
- Глобально диаграммы делятся на два вида: структурные и поведенческие. Структурные описывают, из каких элементов состоит система, а поведенческие — как она себя ведёт внутри.
- А ещё структурные и поведенческие диаграммы делятся внутри на 14 более специфичных диаграмм, где у каждой своё предназначение.